JetBot 連線 SOP（詳細版）

適用環境：MacBook Pro（M4 Pro）+ USB-C 轉 RJ45 Hub + Jetson Nano JetBot JetBot 資訊：主機名稱 `nano-4gb-jp45`，Ubuntu 18.04.5 LTS（L4T 4.9.201-tegra，aarch64），SSH 帳號 `jetbot`，網卡 MAC `48:b0:2d:5c:2e:9b`

一、硬體元件辨識

JetBot 主要由兩塊板子疊組而成：

Jetson Nano 主板（Carrier Board）

整台機器裡**最大塊**的電路板，印有 NVIDIA 標誌

板上元件：

Barrel Jack：黑色圓形直流電源孔，中間有一根小金屬柱突出，位在板子邊緣、靠近網路孔跟 HDMI 孔那一側

J48 跳線帽：位在 Barrel Jack 正旁邊，兩根針腳（2-pin header），套上跳線帽可切換供電來源

網路孔（RJ45）：跟 Barrel Jack 同一塊板子上

HDMI、**USB 孔**

microSD 卡插槽

散熱風扇

D53 LED：Jetson Nano 核心模組（Compute Module）本身的 Power Good 指示燈，跟主板上另一顆綠色電源燈是不同顆、代表不同意義

馬達板（Motor Driver Board）

比 Nano 小，通常疊在 Nano 下方或旁邊

接著左右輪馬達的線（紅黑或四芯線）

可能有獨立電池座，也可能與 Nano 共用電源

上面有 **PWR**、**STA** 等狀態指示燈

快速辨認技巧：順著馬達的接線往回追，接到的那塊板子就是馬達板。

二、供電模式與 J48 跳線帽

Jetson Nano 有兩種供電方式，由 J48 跳線帽決定：

J48 狀態	供電來源	建議規格
沒有跳線帽（預設）	Micro-USB	5V / 2A（僅供 Nano 模組本身，不足以應付完整週邊+馬達）
套上跳線帽	Barrel Jack	5V / 4A（官方建議，供整塊板子+週邊使用）

J48 位置：在 Barrel Jack 旁邊，跳線帽本身**沒有正反面之分**（只是短路兩根針腳的塑膠帽，不涉及極性），但務必**插到底**，確保兩根針腳都確實被內部金屬夾片接觸到。

⚠️ **已知問題**：這次排查中曾遇到 J48 跳線帽**插到底反而接觸不良、要半插著才能開機**的情況。這是接觸點本身有瑕疵的警訊（可能針腳氧化、變形，或跳線帽夾片彈性不足），不建議長期用半插方式應急，因為之後只要有輕微震動（機器人移動、馬達運轉）就可能瞬間斷電，導致系統當機或 SD 卡損毀。長期使用建議換一顆新的跳線帽，或檢查/扳正針腳。

三、電源相關故障模式與排查邏輯

現象 A：風扇轉、綠燈亮，但網路孔（及其他週邊）不亮

原因：Jetson Nano 的電源管理機制在偵測到電流不足或電壓不穩時，會**主動關閉週邊裝置的供電**（USB、HDMI、Ethernet）來保護核心晶片，但核心運算跟風扇因為耗電量小，仍可以正常運作。這是已知的官方論壇常見案例，不是操作錯誤。

常見成因：

電源供應器規格不足（即使標示 5V/4A，如果同時要供應馬達板，總電流需求會超出）

電源線材/接頭品質不佳造成壓降

電源本身輸出不穩定（尤其是行動電源，負載增加時電壓容易下降到 4.7V 以下）

排查方法：

用電表量測 Barrel Jack 輸入端實際電壓，確認是否維持在 4.75V 以上

換一顆確定規格無虞的電源測試

關鍵測試：暫時拔除馬達板的電力連接，只讓 Nano 單獨使用該電源，觀察網路孔是否恢復正常。若恢復正常，證實是電流被馬達板分走導致。

現象 B：風扇轉一下就停，綠燈仍亮

原因：通常是接觸不良（例如 J48 跳線帽鬆動），造成間歇性斷電，而不是電源功率不足本身的問題。

排查方法：

檢查 J48 跳線帽是否插到底、針腳有無氧化彎曲

重新插拔測試

現象 C：換成標配 5V/4A 直流電源後問題依然存在

已確認的根本原因：**馬達板跟 Nano 共用同一顆電源**。5V/4A 是 Jetson Nano「單獨供電」時的建議規格，如果同一顆電源還要分流給馬達板，兩邊加總的電流需求會超過供應能力，尤其在開機瞬間或馬達啟動的尖峰負載時，容易造成電壓下降、觸發保護機制關閉網路孔等次要週邊。

解決方向：

分開供電（建議）：Nano 用電源專用，馬達板另外接獨立電源（如電池組）

換更大電流的電源：如果設計上就是要共用一顆電源，換成 5V/6A 以上

⚠️ 重要安全提醒：不要讓兩個獨立電源同時透過 GPIO 排針連接

如果馬達板跟 Nano 之間除了訊號線，還透過 **40-pin GPIO 排針**傳輸電力（很多 JetBot 套件是這樣設計，兩塊板子疊接、共用同一路電力），**絕對不要**讓馬達板跟 Nano 分別接上兩個獨立電源（例如馬達板接行動電源、Nano 同時接 Barrel Jack）。這樣會造成兩個電源透過共用的 GPIO 排針互相「打架」，有電流反灌、損壞元件的風險。

若不確定馬達板跟 Nano 之間是否共用電力，先目視檢查兩者之間的排線：

若是**整排（40 根）疊接**：很可能共用電力

若只有**幾條細杜邦線**：通常只是訊號線，不含電力（但仍建議確認是否含 5V/GND）

在確認清楚之前，**一次只接一個電源測試**，不要同時通電。

四、安全斷電與換電源順序

斷電順序

若系統可連線操作（已知 IP、能 SSH），先執行：

```
重擊
```

```
sudo shutdown -h now
```

等待風扇停止、所有燈號熄滅，確認完全斷電，避免 SD 卡寫入中斷造成資料損毀。 2. 若無法連線操作，直接斷電，順序為：**先拔/關閉馬達板電源 → 再拔 Nano 電源**。

邏輯：不管兩者是否共用電源，這個順序都安全——若獨立，本來就沒風險；若共用，拔馬達板時 Nano 也會跟著斷電，達到「完全斷電後才切換」的效果。

切換電源來源

確認舊電源完全拔除、板子完全斷電（風扇停、燈滅）

確認新電源**本身尚未通電**（插座開關關閉、或電源供應器沒插電）

把新電源接頭插入 Barrel Jack，確實插到底

確認 J48 跳線帽仍然穩固接著

這時候才把新電源接上插座、打開開關，讓電真正流進去

開機後檢查：風扇是否持續轉動（不是轉一下就停）、綠燈是否穩定、網路孔是否亮起

最後才視需要接上馬達板電源（避免馬達在系統未開機完成時意外通電轉動）

核心原則：先拔電源、後接電源時「先接好接頭、再通電」，避免插入瞬間的電流突波損傷板子。

五、實體網路連線

網路線一端接 JetBot 的 RJ45 孔，另一端接 MacBook 的 USB-C 轉 RJ45 Hub

這是**直連（點對點）**，中間沒有經過路由器或交換器

直連狀態下，雙方都沒有 DHCP 伺服器可以分配 IP，這點會影響下一步的設定方式

如何判斷是直連還是區網

看網路線的另一端有沒有再接到路由器/交換器/牆上網路孔：

只有 MacBook → Hub → 網路線 → JetBot，中間沒有經過其他裝置 → **直連**

網路線是從路由器/交換器拉出來接到 JetBot，MacBook 也連到同一台路由器 → **區網**

六、macOS 端網路介面判斷

常見混淆點：多個 en 介面

USB-C Hub 可能會在 macOS 上模擬出多個網路介面（例如 en4、en5、en6、en7），需要逐一排查找出真正對應 Hub 的那一個：

```
重擊
ifconfig en4
ifconfig en5
ifconfig en6
ifconfig en7
```

每個介面查看兩個關鍵欄位：

`status: active` 還是 `inactive`（判斷實體連線是否建立）

`media:` 是否顯示類似 `1000baseT <full-duplex>`（代表偵測到真正的網路訊號，而非 `media: none`）

本次案例中，實際對應 Hub、且狀態正常的介面是 **en7**（`status: active`，`media: autoselect (1000baseT <full-duplex>)`），en4/en5/en6 皆為 `inactive`、`media: none`（可能是 Hub 的其他埠，或未使用的介面）。

常見誤區：預設路由介面干擾

macOS 執行 `ping`、`arp` 等指令時，預設會透過**有網際網路路由的介面**（通常是 en0，例如連到 WiFi 或校園網路），完全繞過接 JetBot 的那個介面，導致 `arp -a` 只看到校園網路相關的設備、或路由器的 Proxy ARP 假回應

（同一個 MAC 對應所有查詢的 IP，這是企業/校園網路常見的防護機制）。

解法：明確指定介面：

```
重擊
ping -b en7 -c 2 169.254.135.1
```

七、讓 JetBot 透過 Mac 拿到正常 IP（關鍵步驟）

為什麼需要這一步

直連狀態下沒有 DHCP 伺服器，macOS 會自動退回使用 **169.254.x.x（APIPA / Link-Local Address）**這種臨時位址；但 **Linux（JetBot 系統）預設不會自動這樣做**——如果 JetBot 一直在等 DHCP 回應但等不到，可能就卡在沒有 IP 的狀態，不會像 Mac 一樣自動生一個 169.254.x.x。這也是為什麼 Mac 端能看到自己的臨時 IP，但完全掃描不到 JetBot 的原因。

解法：開啟 macOS 網際網路共享，讓 Mac 扮演 DHCP 伺服器

系統設定 App → 一般 → 共享

點選「網際網路共享」

共享連線來源：選 `Wi-Fi`（⚠️ 方向不要選反——選 Wi-Fi 代表「把 Wi-Fi 的網路分享出去」，如果誤選成 `USB 10/100/1000 LAN` 會變成方向相反）

裝置使用清單：勾選 `USB 10/100/1000 LAN`（也就是接 JetBot 的那個介面，對應到 en7）

其他項目（en4、en5、en6、Thunderbolt 橋接器）保持不勾選

點「完成」

回到共享設定主畫面，打開「網際網路共享」最上方的**總開關**

等待約 1 分鐘，讓 JetBot 有時間透過 DHCP 拿到新 IP

驗證共享是否生效

macOS 開啟網際網路共享後，實際的 IP 分配會透過一個叫 **`bridge100`** 的橋接介面管理（不是直接在 en7 上）：

```
重擊
ifconfig bridge100
```

正常會看到類似：

```
inet 192.168.2.1 netmask 0xffffff00 broadcast 192.168.2.255
```

（`192.168.2.1` 是 Mac 自己在這個橋接網路裡的位址，`192.168.2.0/24` 是這次案例中 macOS 網際網路共享預設分配的網段）

同時可以在 `Address cache` 區塊看到透過 en7 學到的裝置 MAC 位址，這通常就是 JetBot 主動發出 DHCP 請求時被偵測到的。

八、找出 JetBot 的實際 IP

方法一：網段掃描（推薦，最可靠）

注意：macOS 的 `ping -t` 是設定 TTL（封包存活次數），不是逾時秒數；正確的逾時參數是 `-W`（單位毫秒）。

序列版（較慢，約 3-4 分鐘）：

```
```

重擊
```
for i in {1..254}; do ping -b en7 -c 1 -W 200 192.168.2.$i > /dev/null 2>&1 && echo "192.168.2.$i is up"; 完成
```
平行版（推薦，數秒內完成）：

重擊
```
for i in {1..254}; do (ping -c 1 -W 200 192.168.2.$i > /dev/null 2>&1 && echo "192.168.2.$i is up") & 完成; wait
```
跑完後會列出所有有回應的 IP，扣掉 Mac 自己（`192.168.2.1`），剩下的就是候選對象。

方法二：確認 MAC 位址對應

掃描結果找到候選 IP 後，確認其 MAC 位址與已知的 JetBot 網卡 MAC 一致：

重擊
```
arp -a | grep 192.168.2.3
```
（JetBot 已知網卡 MAC：`48:b0:2d:5c:2e:9b`，本次案例中固定落在 `192.168.2.3`，但實際 IP 可能因每次 DHCP 分配而略有不同，建議每次都重新確認）

方法三：mDNS 主機名稱（不一定可靠）

重擊
```
ping jetbot.local
ping nano.local
```
⚠ 此方法在本次案例中失敗（顯示 `Unknown host`），可能是 JetBot 的 mDNS（Avahi）服務尚未啟動完成，或主機名稱不是預設值。**不建議依賴此方法**，優先使用網段掃描。

九、SSH 連線

重擊
```
ssh jetbot@192.168.2.3
```
（IP 依實際掃描結果替換）

帳號：`jetbot`

第一次連線會出現：

```
The authenticity of host '192.168.2.3 (192.168.2.3)' can't be established.
...
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```
輸入 `yes` 後按 Enter，此為正常的首次連線信任提示。

接著會要求輸入密碼，終端機不會顯示任何字元（含游標），這是正常的安全機制，直接打完密碼按 Enter 即可。

成功後會看到類似：

```
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 4.9.201-tegra aarch64)
jetbot@nano-4gb-jp45:~$
```

十、確認 JetBot 端網路狀態（雙向驗證）

登入 JetBot 後，反過來確認網路介面狀態，跟 Mac 端看到的資訊互相對照：

```
重擊
ifconfig eth0
```

應該看到：

`inet` 位址跟 Mac 端 `arp -a` 查到的一致

`ether` 後面的 MAC 位址跟已知的 JetBot 網卡 MAC 一致

`RX errors`、`TX errors`、`dropped` 都是 0，代表連線品質正常，沒有封包遺失

十一、開啟 Jupyter Notebook / JupyterLab

背景知識

JetBot 官方映像檔（NVIDIA-AI-IOT JetCard）是用 **Docker 容器**跑 Jupyter，而不是直接把 Jupyter 裝在系統上，所以系統層級找不到 `pip3`、`jupyter` 指令是正常的，不代表軟體沒裝好。

容器資訊：

容器名稱：`jetbot_jupyter`（另外還有一個 `jetbot_display` 容器）

映像檔：`jetbot/jetbot:jupyter-0.4.3-32.5.0`

網路模式：`--network host`（容器直接共用主機網路，不需要額外 port mapping，8888 port 直接開放在主機 IP 上）

對應原始碼位置：`/home/jetbot/jetbot/`（JetBot 官方 GitHub repo）

另有 `/home/jetbot/jetcard/` 資料夾（NVIDIA JetCard 基礎映像的一部分）

檢查容器狀態

```
重擊
sudo docker ps -a
```

正常應該看到 `jetbot_jupyter` 跟 `jetbot_display` 兩個容器狀態為 `Up`。

若需要進一步排查監聽狀態

確認 port 有沒有正確映射：

```
重擊
sudo docker inspect jetbot_jupyter | grep -A 10 "PortBindings"
sudo docker inspect jetbot_jupyter | grep NetworkMode
```

確認主機本身是否真的在監聽 8888：

```
重擊
```

```
sudo netstat -tlnp | grep 8888
# 或
sudo ss -tlnp | grep 8888
```

正常應顯示：

```
tcp   0   0 0.0.0.0:8888   0.0.0.0:*   LISTEN
```

瀏覽器開啟

在 Mac 瀏覽器輸入：

```
http://192.168.2.3:8888/lab
```

（IP 依實際掃描結果替換；直接帶 `/lab` 路徑，因為伺服器預設會用 302 轉址導向 `/lab`）

⚠️ **已知問題**：用**無痕視窗（Incognito）**開啟時曾出現 `ERR_ADDRESS_UNREACHABLE` 錯誤（瀏覽器層級的路由問題，不是伺服器問題——用 `curl` 測試同一個網址是完全正常的）。**解法：改用一般（非無痕）瀏覽器視窗**即可正常連線。

排查用：用 curl 驗證伺服器本身是否正常（不透過瀏覽器）

```
重擊
curl -v http://192.168.2.3:8888
```

正常會看到類似：

```
< HTTP/1.1 302 Found
< Server: TornadoServer/6.1
< Location: /lab?
```

若 curl 正常但瀏覽器連不上，問題出在瀏覽器端（無痕模式限制、DNS 快取等），不是伺服器或網路問題。

十二、常見問題排查對照表

現象	可能原因	排查方式
風扇轉、綠燈亮，但網路孔（及其他週邊）不亮	電源電流不足，電源管理機制關閉週邊供電（尤其馬達板共用電源時）	用電表量測電壓；暫時拔除馬達板電力，只讓 Nano 單獨用電源測試
風扇轉一下就停，綠燈仍亮	J48 跳線帽接觸不良，間歇性斷電	檢查跳線帽是否插到底、針腳有無氧化彎曲；必要時更換跳線帽
換成標配 5V/4A 電源仍有問題	馬達板與 Nano 共用同一顆電源，總電流需求超出供應能力	分開供電，或換更高電流（如 5V/6A）電源
馬達板與 Nano 分別接兩個獨立電源，出現異常閃爍	若兩者透過 GPIO 排針共用電力，雙電源同時供電會互相干擾、有反灌風險	立即斷電；確認排線方式後只用單一電源供電
Mac 網路孔顯示「未連線」、`status: inactive`	網路線鬆脫、走錯介面、Hub 相容性問題	重新插拔；逐一檢查 en4/en5/en6/en7 等介面找出正確的一個
`arp -a` 只查到自己或校園網路裝置	指令透過錯誤介面（如 en0）發送，或遇到路由器 Proxy ARP 假回應	用 `-b <介面>` 明確指定介面
Mac 有 IP（169.254.x.x）但完全掃不到 JetBot	Linux 不像 macOS 會自動退回 APIPA，JetBot 可能卡在沒有 IP 的狀態	開啟 macOS 網際網路共享，讓 Mac 扮演 DHCP 伺服器
`ping jetbot.local` / `nano.local` 找不到主機	mDNS 服務未啟動或主機名稱非預設值	改用網段掃描 + MAC 位址比對，不依賴主機名稱解析
SSH 可連但瀏覽器連不到 8888（`ERR_ADDRESS_UNREACHABLE`）	無痕視窗對本地網段連線的限制、瀏覽器快取	用 `curl` 驗證伺服器正常後，改用一般（非無痕）視窗重試
系統裡找不到 `pip3`、`jupyter` 指令	JetBot 用 Docker 容器跑 Jupyter，非直接安裝在系統上	用 `docker ps -a` 確認容器狀態，不必額外安裝

十三、每次開機的建議標準流程（濃縮版）

檢查 J48 跳線帽插緊、電源規格足夠（若馬達板共用電源，優先分開供電）
先接好所有接頭，最後才通電
開機後等待 1~2 分鐘，觀察風扇是否持續轉動、網路孔是否亮起
確認 macOS「網際網路共享」設定仍然開啟（USB 10/100/1000 LAN 有勾選）
`ifconfig bridge100` 確認 Mac 端橋接介面正常
掃描網段找出 JetBot IP（或先嘗試已知的 `192.168.2.3`）

SSH 連線確認系統正常

瀏覽器（一般視窗）開啟 `http://<IP>:8888/lab`